

ScalaLR

ScalaLR is a straightforward LR(1)-parser generator for Scala that translates its own host notation (a description of a target notation) to the essential components of a bottom-up parser for the target notation expressed in Scala.

Its accompanying library provides implementations both of `Pull` and of `Push` parsing automata that are used (with the parsing tables it generates) to implement the target language parser. The former is designed for conventional "parse-to-completion" situations, the latter supports the engineering of incremental parsers whose state and intermediate results can be inspected "in flight".

Getting started (users)

The current version of the tool, together with all the runtime libraries a generated parser needs is packaged in a java-runnable jar in:

```
scalalr.jar
```

Run it with

```
java -jar scalalr.jar [arguments]
```

(or an equivalent script). The same jar should be on the classpath when compiling or running a generated parser.

Requirement: a `bison` (GNU Bison) 3.8.2 (or a later consistent version) must be accessible on the current path. We use only the publicly documented features of Bison for generating tables.

Host notation

The host notation for grammar productions and priorities is reminiscent of Bison's notation; but there are important overall differences from Bison, as exemplified by the following fragment. We

will document these in detail in due course.

```
%notation Expr // $0
%package expr.Expr // $0
%tables lalr // $0

%include {
  // Scala source to be included in a generated
  // file that supports or implements a lexer
  import org.sufrin.utility.SourceTextCursor
  import org.sufrin.scalalr.SourceLocation

  def Scanner(chars: SourceTextCursor): Scanner = new Scanner(chars)

  class Scanner(chars: SourceTextCursor) extends Iterator[Token] { ... }
}

%token ID(String) `(` `)` `[` `]` `;` LEXICALERROR(String) // $1
%left `+` // $2
%left `*` // $2
```

0. All notations have a name, and Scala code is generated with a package prefix that (by convention) ends in that name. The target notation parse tables used are generated as (canonical) *LR* tables, as *LALR* tables, or as *IELR* tables.
1. Tokens (terminal symbols) *must* be specified. Each that carries an irredundant value must have the type of that value specified.
2. Shift-reduce conflicts can be resolved by specifying the associativity and precedence of (terminal) symbols, as in Bison.

```
%rules

%include {
  import org.sufrin.scalalr.SourceLocation
  // Scala source to be included in a generated file
  // that supports or implements the abstract syntax
  // (or other values) specified as production values

  trait Expr { val loc: SourceLocation } // $4
  case class Id(s: String, loc: SourceLocation) extends Expr
  case class Binop(op: String, l: Expr, r: Expr, loc: SourceLocation) extends Expr
  case class Bra(expr: Expr, loc: SourceLocation) extends Expr
}
```

```

exprs: (List[Expr]) = expr          { List($expr) }           // $3, $4
                        | exprs `;` expr { $expr::$exprs }     // $4

expr: Expr = ID                    { Id($ID, $START) }       // $4,
    | l:expr `*` r:expr            { Binop("*", $l, $r, $START) }
    | l:expr `+` r:expr            { Binop("+", $l, $r, $START) }
    | "(" expr ")"                 { Bra($expr, $START) }     // $5
    | `[` expr `]`                 { $expr }

```

3. Nonterminal symbols have types specified explicitly on the left hand side of their definition.
4. The abstract syntax node (or other value) represented by each production is specified as a Scala block expression at its end. Such expressions may refer to the values of symbols (terminal or nonterminal) that appear in the production, by `$label` (for a symbol labelled in the production by prefixing it with `label:`), or by `$symbol` when that `symbol` appears unlabelled and uniquely. They may also refer to the start and end source location of the text matched by the production using `$START` and `$END`.
5. Tokens enclosed in single quotes, double quotes or backticks are treated identically during code generation: they need not be declared in a `%token` section.
6. `%rules` are (these days) separated by visible vertical space, or by semicolons followed by any amount of empty space.

The generated target language code can easily be incorporated into a production Scala program. Here's a test of the earlier example that uses the `Pull` automaton.

```

object runexpr {

  import expr.Expr.Components
  import expr.Expr.Scanner._

  import org.sufrin.utility.PrettyPrint._
  import org.sufrin.utility._

  def main(args: Array[String]): Unit = {
    val source = """a; a+b; a*b+c*d*(e+f)*[g+h]; p+q*r"""
    val scanner = Scanner(SourceTextCursor(source))

    def next(): Token = if (scanner.hasNext) scanner.next() else $end

    val parser = LRParser.Pull[Token](Components)(scanner.sourceLocation)
    parser.run(next).prettyPrint()
  }
}

```

}

Documentation is evolving but for the moment it should be sufficient for a knowledgeable reader to inspect the source texts of the notation specifications and driver programs to be found within directories nested within `examples`.

Implementations

Grammar analysis

ScalaLR uses **GNU Bison (version 3.8.2)** as an internal workhorse to compute the LR shift-reduce parser tables for the grammar (rules) of the source notation, and to do any necessary detailed diagnostics on the grammar.

Production-quality Implementations

There are (now: early May 2026) several stable production quality implementations of the program, bootstrapped from an original handwritten parser and a simple code generator.

1. The `scalalrboot` program uses the original bootstrap handwritten parser and the original bootstrap code generator. Guess why we called it `scalalrboot` ...

» Usage: `scalalrboot [--output=<outputpath>] [ <file> ...]`

2. The `scalalrstage1` program has a notation-description language that is lot like the bootstrap generator.

» Usage: `stage1 [--output=<outputpath> (default STAGE10OUTPUT) | -o <outputpath>]`

2. The `scalalrstage2` program has a very forgiving notation-description language, and many more options than its predecessor. More importantly it generates fairly good error diagnostics: both when something is wrong with the notation description it is working on, and also when the grammar rules give rise to LR parsing conflicts.

» Usage: `scalalrstage2 OPTION ... PATH ...`
Treat (each) `PATH` as the path `in` the filestore to `scalalr SOURCE` and gene

scala files corresponding to the %notation it defines.

Place the generated files under the directory named by OUTPUTPATH catenated with the %path (if any) declared in the scalar source. The default OUTPUTPATH is `"./generated"`.

OPTIONS:

-pp prettyprint only
-log log the input source parse
-html output grammar report in html form
-c generate detailed conflict report (with "counterexample" derivations -- these can take a long time to generate, and timeouts operate)

LOGGING OPTIONS

-Lsym show an inventory of the symbols, their types, and their definitions
-Lsyn show the rules after code synthesis for repeated constructions`

OUTPUTPATH is set by one of

-p OUTPUTPATH
-o OUTPUTPATH
--output=OUTPUTPATH
--prefix=OUTPUTPATH

LITERAL SOURCE (reserved for programmatic testing) a notation may be defined

-# INT first SOURCE line number
-## INT first SOURCE column number
-s SOURCE

Generated files

When all is well, the generator produces several components (in distinct files) from each parser specification; these appear in the directory corresponding to %path specification, and are named:

1. **Scanner.scala** defines the Scala case classes and types corresponding to each %token definition. There is provision for including the source text of the lexical scanner in this file.
2. **Tables.scala** defines the shift-reduce tables corresponding to the grammar. These will be interpreted by an LRParser automaton at parse-time.
3. **Reduction.scala** defines a function that maps each production number to a function that combines the values that are generated by its right-hand-side to form the value of the production itself.

4. **Components.scala** defines a structure that incorporates the semantics of the Tables and Reduction files.

These files are generated in four phases:

0. The generator performs a superficial sanity check on the notation description, and informs of any obvious problems. If this is successful...
1. The generator produces a plain Bison grammar `.y` file in which all grammar symbols (terminal and nonterminal) have been transformed to straightforward Bison names of the form **TOK-*ddd*** in order to avoid confusing Bison. Under normal circumstances one need not inspect the `.y` file, and symbols in the diagnostic/report files are expressed as they were in the Scalalr source.
2. Bison is run to generate the LR parse tables as well as report and diagnostic files, all named for the specified `%notation`. The `.output` and `.html` files contain identical information: a report on the notation with details of states, conflicts, etc. The `.log` file provides (when appropriate) counterexamples illustrating conflicts in the grammar. These are intended to be helpful in diagnosing conflicts/ambiguities that Bison discovers while processing the grammar specification.

Files *notationName*.{xml,output,log,y}

1. The `.xml` file output by Bison contains, among other information, an encoding of the information Scalalr now uses to generate its **Tables** and **Reduction** files.
2. The `.output` file output by Bison contains a full report on the parsing automaton, including details of any conflicts.
3. When output the `.html` file output by Bison contains an *easily navigable* full report on the parsing automaton, including details of any residual conflicts. The `stage2` generator enables this with the `-html` flag: earlier stage generators always enable it.
4. The `.log` file output by Bison may contain detailed diagnostic messages that help explain conflicts.

Repetition Notations

The *stage2* and subsequent processors include the repetition modifiers `*`, `+` and `?` that make it straightforward to express repeated and optional constructs "in-situ" rather than having to write the grammar rules for them explicitly.

(A)*	means zero or more A and yields the List[A] of their values
(A)+	means one or more A and yields the List[A] of their values

(A)? means zero or one A and yields the appropriate Option[A]

When B is not a value-carrying symbol, the repetition constructs denote "punctuated" sequences and yield the List[A] of the value-carrying symbol

(`B` A)* means zero or more A separated by `B`
(`B` A)+ means one or more A separated by `B`
(A `B`)* means zero or more A separated by `B`
(A `B`)+ means one or more A separated by `B`

For example

```
...
%rules
  expr: Expr = ID '(' exprlist: (',' expr)* ')' { Apply($ID, $exprlist) }
        |    '{' exprlist: (';' expr)+ '}' { Sequence($ID, $exprlist) }
        |    RETURN optexpr: (expr)? ';' { Return($optexpr) }
```

will be transformed into invocations of additional, mechanically-derived, rules (with mechanically generated names)

```
001  expr: Expr      = ID '(' exprlist: S_1 ')' { Apply($ID, $exprlist) }
002  expr: Expr      =   '{' exprlist: S_2 '}' { Sequence($ID, $exprlist) }
003  expr: Expr      =   RETURN optexpr: S_3 ';' { Return($optexpr) }
004  S_1_L: List[Expr] = expr { List($expr) }
005  S_1_L: List[Expr] = S_1_L ',' expr { $expr :: $S_1_L }
006  S_1: List[Expr]  = S_1_L { $S_1_L.reverse }
007  S_1: List[Expr]  = %empty { Nil }
008  S_2_L: List[Expr] = expr { List($expr) }
009  S_2_L: List[Expr] = S_2_L ';' expr { $expr :: $S_2_L }
010  S_2: List[Expr]  = S_2_L { $S_2_L.reverse }
011  S_3: Option[Expr] = %empty { None }
012  S_3: Option[Expr] = expr { Some($expr) }
```

Notice the left-recursive productions for list-yielding constructs. These conserve parse-stack space and list-construction time at the cost of accumulating their result lists in reverse order, then reversing them at the point of use.

Notice that in general it is appropriate to provide a name for the result of a repeated construct, so that it can be embodied in the result expression of its "host" production.

NB: The repetition constructs are for convenience in simple cases: if one needs more than just straightforward "punctuated list" expansions in a production it is better to hand-code the additional rule(s)

Further Reading

1. The best-documented simple examples of programs that uses *scalalr*-generated parsing components are in directories under `examples/`.
2. The file `Bootstrapping.md` provides an explanation of the self-hosting bootstrap stages.
3. **Beware:** Some quick'n'dirty programs that were used in building and testing the initial bootstrap appear in `bootstrap/src/test`. They generate(d) components in the **testbed** module's `src/test/scala` directory. They may mislead you.

LR Parsing Conflict Illustrations

A few notation definitions that result in shift-reduce or reduce-reduce conflicts are gathered (as embedded strings) in the `GeneratorTests` of `stage2`. Here's an ambiguous grammar, and the full diagnostic reported by the `stage2` generator with `-c set`.

```
%token a
%rules
S = A
  | B;
A = a;
B = a;
```

```
TEST-GENERATED/conflicts/SAB/SAB.y: warning: 1 reduce/reduce conflict [-Wconfl
TEST-GENERATED/conflicts/SAB/SAB.y: warning: reduce/reduce conflict on token $
Example: a •
First reduce derivation
S
↳ 1: A
↳ 3: a •
Second reduce derivation
S
↳ 2: B
↳ 4: a •
```

Here's the classic "dangling else" ambiguity, and a full diagnosis

```
%token IF THEN ELSE ID '+'
%rules
expr = ID
    | expr '+' ID
    | IF expr THEN expr
    | IF expr THEN expr ELSE expr
```

TEST-GENERATED/conflicts/IfThenElse/IfThenElse.y: warning: 3 shift/reduce co
 TEST-GENERATED/conflicts/IfThenElse/IfThenElse.y: warning: shift/reduce conf

Example: IF expr THEN expr • '+' ID

Shift derivation

```
expr
↳ 3: IF expr THEN expr
                                ↳ 2: expr • '+' ID
```

Reduce derivation

```
expr
↳ 2: expr                                '+' ID
    ↳ 3: IF expr THEN expr •
```

TEST-GENERATED/conflicts/IfThenElse/IfThenElse.y: warning: shift/reduce con

Example: IF expr THEN IF expr THEN expr • ELSE expr

Shift derivation

```
expr
↳ 3: IF expr THEN expr
                                ↳ 4: IF expr THEN expr • ELSE expr
```

Reduce derivation

```
expr
↳ 4: IF expr THEN expr                                ELSE expr
    ↳ 3: IF expr THEN expr •
```

TEST-GENERATED/conflicts/IfThenElse/IfThenElse.y: warning: shift/reduce con

Example: IF expr THEN expr ELSE expr • '+' ID

Shift derivation

```
expr
↳ 4: IF expr THEN expr ELSE expr
                                ↳ 2: expr • '+' ID
```

Reduce derivation

```
expr
↳ 2: expr                                '+' ID
    ↳ 4: IF expr THEN expr ELSE expr •
```

Gotchas

1. **Error recovery** is not yet satisfactorily implemented. Both `Pull` and `Push` automata report the first syntax error and bail by throwing an exception, unless their `attemptRecovery` variable is `true` and an `error`-handling state is present on the parse stack (see the Bison documentation for an explanation of the `error` virtual token). If their `logRecovery` variable is `true` then the recovery attempt is documented (in some detail). It is nevertheless straightforward to construct a Read-Eval-Print-type interface that appears to recover from syntax errors by enclosing its top-level in a throw-handling loop.
2. **Trivial typos** used to be problematic. Experience with using the notation-specification languages early in the bootstrap sequence *used to* demonstrate the high incidence of errors caused by the omission of semicolons between rules. This was *corrected in and after stage1: a semicolon can be omitted between two rules, providing there is visible vertical space between them*. Semicolons are still forbidden between `%...` directives before `%rules`.
3. **Scala code quotations** such as appear in `%include` passages and as production result expressions need a little care. The normal form of a code quotation is a passage that opens with `{`, has properly-nested occurrences of `{` and `}` within it and ends with a closing `}` that matches the opening. **But** if an unmatched brace appears (for example in a character or string quote or in a comment) it can upset balance and lead to an incorrect analysis.
 - i. As it happens *Bison/Yacc themselves have analogous (not identical) lexical requirements for code inserts*. The only squeaky-clean solution to this kind of thing is to build a parser for the target language of the generator within the generator; and this is not really practical for Scala (given our resources).
 - ii. **Happily** almost all the potential pain is avoidable: just use double-braces to start and end the quotation; and don't worry about internal non-balance.
`{{ like { th{is }}`
 - iii. **and** if you *must* quote double braces, do it within guillemot-brackets
`« like {{ th{{is }}`

Roadmap

We aim to accomplish the following tasks as soon as we can. They are listed here in no particular order.

1. Error recovery (in generated parsers) properly implemented.

2. **[DONE April '26]** System to be self-hosting: ie using a scalar-derived parser rather than the present hand-coded recursive descent parser.
3. **[DONE May '26]** Implement higher-level constructs for use in-situ in productions to denote repetitions and options that would normally have to be "hand-coded".
4. Additional higher level constructs that support "say it once" specification of notation and abstract syntax, as well as less "fragile" expressions of the AST results of productions. We have in mind doing this in two stages. The first exploits the fact that scala methods can be called with "keyword" parameters. Suppose our expression abstract syntax has been simplified:

```

trait Expr
  case class Id(name: String) extends Expr
  case class Binop(op: String, l: Expr, r: Expr) extends Expr
  case class Bra(expr: Expr) extends Expr

```

then a concise way of expressing Expr yielding productions would use a "result" arrow that would be translated into ordinary code sections, for example:

```

expr: Expr = name: ID          -> Id
      | l:expr op: `*` r:expr -> Binop
      | l:expr op: `+` r:expr -> Binop
      | "(" expr ")"          -> Bra
      | `[` expr `]`          { $expr }

```

and the generator could turn this mechanically into

```

expr: Expr = name: ID          {Id(name=$name)}
      | l:expr op: `*` r:expr {Binop(l=$l, op=$op, r=$r) }
      | l:expr op: `+` r:expr {Binop(l=$l, op=$op, r=$r) }
      | "(" expr ")"          {Bra(expr=$expr)}
      | `[` expr `]`          { $expr }

```

Working Assumption

ScalaLR was designed on the assumption the components it generates will become part of a parser that will yield an abstract syntax tree. *Of course there's no harm in the parser yielding (say) a numeric value, or even being run for its side effects and yielding Unit -- though this may not be common.*

Valediction

The **stage2** code generator, being reasonably-well structured, is now scrutable by others, and might be a good place to start making enhancements to the notation-description language.

I regret the inscrutability of the bootstrap code-generator. I have no excuse for this beyond my having wanted to prioritise fast turnaround while I was first experimenting with my approach. Some might say that the fact that it can **still** be used in a near production environment is testimony to my cunning, but I couldn't possibly comment!

BS: April 29th, May 13th 2026